

Guía formativa curso de Angular



SECCION 1

1. Instalar Angular
2. Crear un proyecto de Angular.
3. Identificar el contenido básico del proyecto
4. Desarrollo en base a Componentes.
 - Creación de nuestras primeras pantallas.
 - i. Creación de componente (Página)
 - ii. La página como conjunto de componentes
2. Navegación básica por una aplicación Angular (router I)
 - Navegar entre paginas
 - Crear una pagina

Al final de este apartado deberíamos ser capaces de:

- Arrancar y crear una nueva aplicación en Angular
- Generar nuevas páginas y componentes
- Navegar entre ellas de forma consistente

SECCION 2

5. Estructurando nuestra aplicación: Los Módulos
 - Generación de módulos
6. Mejoras en la Navegación (router II)
 - LazyLoading
 - RutasHijas

Al final de este apartado deberíamos:

- Poder reorganizar nuestra aplicación para una mejor encapsulación y mantenimiento.
- Separar el fichero de rutas y realizar carga bajo demanda (LazyLoading)

SECCION 3

7. Continuación del uso de Modulos
 - Comunicación entre modulos (import/export)
8. Aplicando lógica dentro de nuestro archivo .ts del componente
 - Propiedades y Signal()
 - Métodos y funciones
 - Estructura de datos: Interfaces
9. Directivas @for, @if en nuestro archivo .html
 - Procesamiento de información dentro de nuestra plantilla html
10. Comunicación entre componentes @input/@output
 - Pasar información del componente padre al hijo (@input)
 - Pasar información del componente hijo al padre (@output)

SECCION 1

1 Archivos básicos en una aplicación en Angular

El diagrama muestra la estructura de archivos de una aplicación Angular en un explorador de archivos. Las carpetas y archivos principales son:

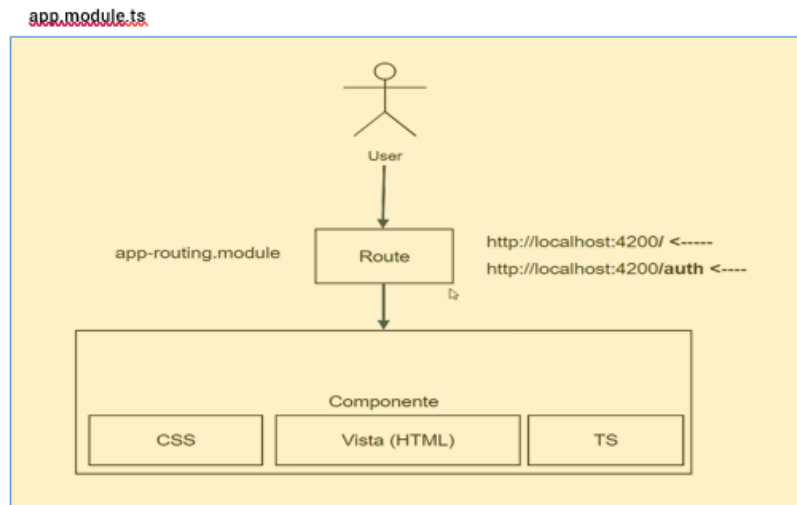
- `intro1` (carpeta raíz)
 - `.angular`
 - `.vscode`
 - `node_modules`
 - `public`
 - `src` (carpeta principal)
 - `app` (componente raíz)
 - `components`
 - `pages`
 - `app.component.css`
 - `app.component.html`
 - `app.component.spec.ts`
 - `app.component.ts`
 - `app.config.ts`
 - `app.routes.ts`
 - `index.html`
 - `main.ts`
 - `styles.css`
 - `.editorconfig`
 - `.gitignore`
 - `.npmrc`
 - `angular.json`
 - `package-lock.json`
 - `package.json`
 - `README.md`
 - `tsconfig.app.json`
 - `tsconfig.json`
 - `tsconfig.spec.json`

Las explicaciones de los archivos y carpetas son:

- Carpeta PRINCIPAL** donde se va a incluir todo el código de nuestra aplicación y donde vamos a trabajar el 99% del tiempo
 - Directorio app: **app** (componente raíz de nuestra aplicación)
- Componente principal aplicación** (desde donde vamos a arrancar)
 - `app.component.ts`
- Fichero principal de rutas de nuestra aplicación** (Navegación)
 - `app.routes.ts`
- Archivos de Configuración de Angular**
 - `index.html` → Pagina de inicio de nuestra aplicación
 - `main.ts` → Punto de entrada de la aplicación. Primer fichero que se carga. Inicializar la app
 - `style.css` → Estilos globales
- Archivos de Configuración de Angular**
 - `Package-lock.json` → **No tocar**. Utilizado para reconstruir nuestro `node_modules` de note
- Archivo Configuración TypeScript**
 - Recomendaciones recomendadas por Angular
 - Se puede editar para evitar algunos warnings/errores
 - + Extensiones para añadir información cuando se está corriendo la app en `prod/test`

- `.public` : recursos estaticos de nuestra aplicación (imágenes, iconos, etc)
- `.nodemodules`: No tocar. Todos los paquetes referenciados en el `package.json`. Muy pesado pero solo una pequeña parte llega a la implementación final
- `.dist`: compilacion. Lo que nosotros subimos a nuestro servidor. Version de produccion
- `.vscode`: Configuraciones del editor de codigo de visual studio code
- `.angular`: uso general para manejar cache
- `.gitignore`
- `.editorconfig` : recomendaciones de angular para nuestro codigo

1.1 Estructura Base de la aplicación

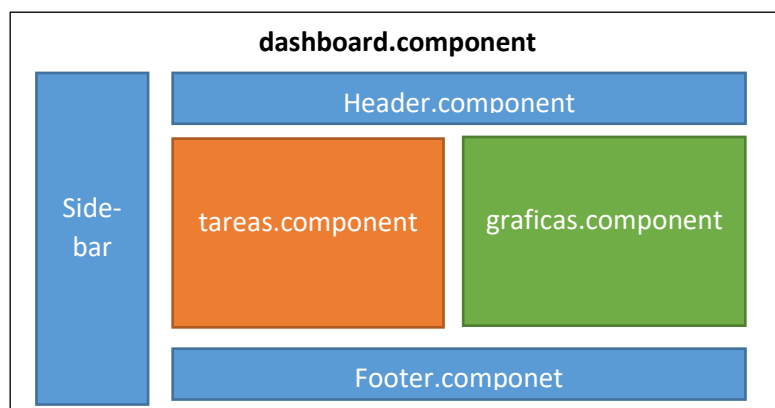


Esquema del arranque

- Inicia por el `index.html`
- Hay tenemos una referencia al `<app-root> = (app.component)` y lo carga en ese punto
- En el `app.component.html` tenemos el `<router-outlet>` que es donde va a renderizar el contenido de la ruta raíz
- Hay mira en el fichero de rutas `app.routes.ts` y cargaría el componente indicado (en este punto vacío)

1 Aplicación Angular: Pensando en componentes

- Podemos decir que los componentes son la base de nuestra aplicación angular.
- Un componente se compone básicamente de 3 archivos + 1 opcional
 - .html
 - .ts
 - .css (podría considerarse también opcional)
 - .spec.ts (opcional)
- Concepto importante!: Cada página de Angular (será un componente) que estará a su vez puede (y debe) estar formada a partir de componentes más pequeños.
 - Tenemos que intentar subdividir en componentes lo más pequeños y específicos posibles para mejorar el mantenimiento y reusabilidad



1.2 Estructura base del componente en código

```
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-header', //identificador del componente para llamarlo en el HTML
  imports: [RouterLink, RouterLinkActive], //Importaciones requeridas por el componente. Ej. Otros componentes
  templateUrl: './header.component.html', //Fichero de plantilla HTML
  styleUrls: ['./header.component.css'] //Fichero de estilos CSS
})
export class HeaderComponent {
  // aquí irá el código propio con la lógica a implementar de este componente
}
```

1.3 Creación de Componentes

- Comando: **<ng g c /path/nombreComponente>**
- Lo va a generar tomado como base a la carpeta **./app**
- Modificadores opcionales más utilizados

OPCION	DESCRIPCION
--skip-tests (*)	No genera el archivo .spect.ts
--inline-template	No genera el archivo de plantilla .html
--inline-style (*)	No genera el archivo de estilos .css
--dry-run	Muestra lo que generaría sin generarlo
--force	Fuerza a sobrescribir si existe el archivo

2 Navegación en Angular. Entender el routing

2.4 ¿Qué es el Router?

El **Router** es el encargado de **gestionar la navegación** entre las diferentes vistas o componentes de una aplicación Angular y determina que componente debe mostrarse para cada URL.

No es **simplemente un sistema para cambiar URLs**: es el mecanismo completo que coordina a través del **URL** → **navegación** → **guards/resolvers** → **carga de componentes** → **renderizado en router-outlet**.

Todo empieza en nuestro componente principal app.component.html, donde encontraremos la etiqueta `<router-outlet />`

Aquí es donde se va a renderizar el contenido que tenga que mostrarse en nuestra aplicación dependiendo de la ruta indicada en la URL

- La idea principal es quitar todo el contenido y dejar el `<router-outlet />` para que renderice el contenido que se tienen que mostrar en esa posición

```
<section>
  <router-outlet />
</section>
```

- También tenemos la posibilidad de que si todas las paginas van a contener elementos comunes como header/footer, etc... dejar las referencias a esos objetos en la html. Ejemplo

```
<main>
  <!-- barra de navegación -->
  <nav><navbar-component /></nav>

  <section>
    <router-outlet />
  </section>
</main>
```

Elementos básicos en el router

Elemento	Función
Router	Gestiona la Navegación
Routes	Definir las rutas disponibles
RouterLink	Permite navegar desde HTML
RouterLinkActive	Marcar la ruta activa
RouterOutlet	Muestra el componente

2.5 ¿Dónde se definen estas rutas?

- La respuesta es en archivo app-routing.module.ts que se encuentra en el directorio raíz de nuestra aplicación.

2.6 ¿Cómo se definen las rutas?

- Vamos a definir un array de objetos donde debemos indicar principalmente el path y el componente que tiene que cargarse si existe coincidencia con ese path

```
import { Routes } from '@angular/router';
import { HomeComponent } from './pages/home/home.component';
import { DashboardComponent } from './pages/dashboard/dashboard.component';
import { AppComponent } from './app.component';

export const routes: Routes = [

  { path: '', component: AppComponent },          // --> ruta raíz localhost:4200
  { path: 'home', component: HomeComponent },      // localhost:4200/home
  { path: 'dashboard', component: DashboardComponent },

  // redirección cuando no encuentra ruta
  { path: '**', redirectTo: 'dashboard' } //alternativa es poner un componente de page not found

];
```

2.7 Navegar desde el componente (RouterLink) y mostrar el link activo (RouterLinkActive)

Por último, tenemos que mencionar como podemos gestionar el link a otras páginas de la aplicación, así como con una forma rápida de marcar cual es la ruta activa

- Con **RouterLink** le indicamos al router que navegue a la ruta indicada
- Con **RouterLinkActive** le podemos decir a angular que automáticamente asigne un estilo a nuestro link activo

```
<a [routerLink]="['/contador']" [routerLinkActive]='active'>Contador</a>
<a routerLink="/personas" routerLinkActive="active">Personas</a>

<!--
• Usa la Opción 1 (Con corchetes []) cuando tu ruta dependa de variables de tu componente.
Ej. Por ejemplo, si el nombre del anime o el ID cambiaran dinámicamente: [routerLink]="['/persona', IdPersonas]"
• Usa la Opción 2 (Sin corchetes) cuando la ruta sea completamente estática, fija y escrita a mano en el HTML
, ya que hace que tu código sea más limpio y fácil de leer.
-->
```

Nota: En este ejemplo, angular asignará la clase 'active' al enlace que este activo. Solo tenemos que definir la propiedad css para la clase .active en nuestro fichero de estilos.

3 Ejercicio base:

1. Generar un componente (mejor si está formado por otros componentes más pequeños)
2. Incluirlo en el fichero de rutas
3. Modificar el componente existente heder para incluir el link al nuevo componente
4. Verificar el funcionamiento

SECCION 2

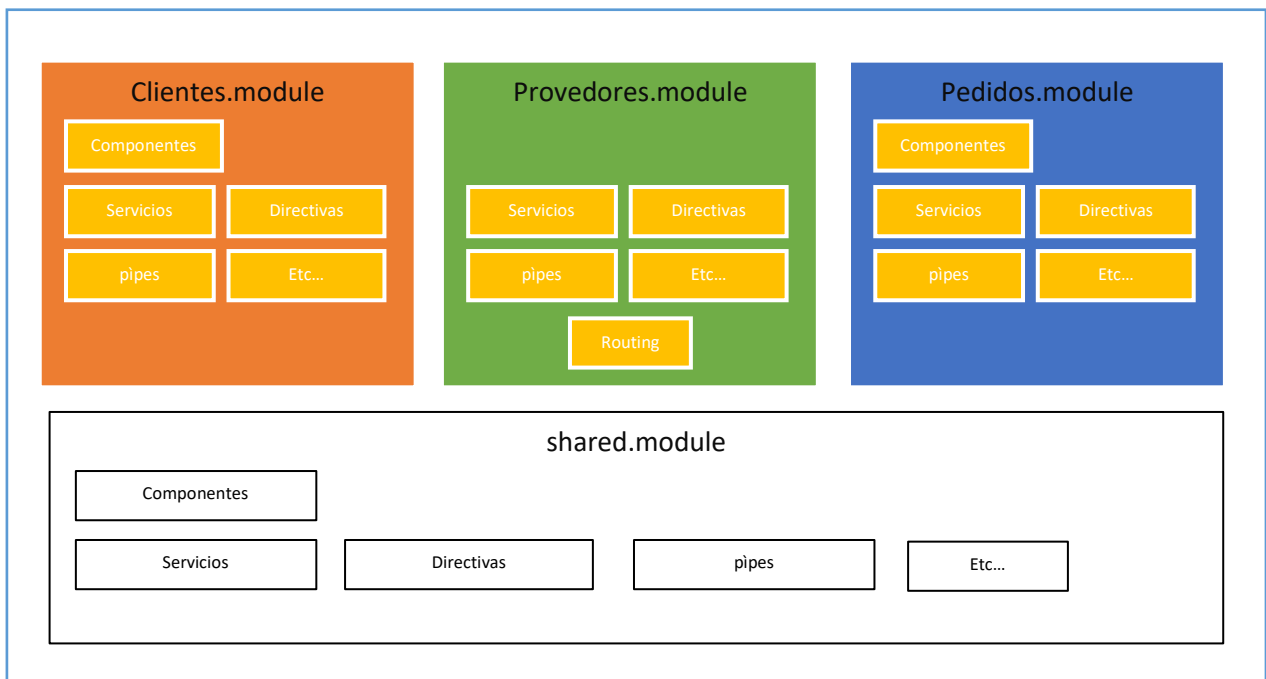
4 Que son los modulos en Angular

Los módulos en Angular se identifican por el uso del Decorador @NgModule y son contenedores lógicos que sirven para agrupar componentes, directivas, pipes y servicios relacionados entre sí.

- Organizan la aplicación: Subdividiendo la aplicación en bloques con funcionalidades relacionadas, más pequeños, ordenados y fáciles de mantener.
- Facilitan la carga bajo demanda (lazyLoading)

4.1 Esquema organizativo mediante modulos.

app.module.ts



NOTA

En las versiones recientes de Angular, el estándar actual son los **componentes standalone** (independientes), los cuales permiten declarar dependencias directamente en cada componente sin necesidad de crear un archivo de módulos.

Estructura de NgModule

- **Declarations:** Componentes, directivas y pipes que pertenecen a este módulo.
- **Imports:** Otros módulos necesarios para que este funcione
- **Providers:** Servicios que estarán disponibles globalmente en el ámbito del módulo.
- **Exports:** Elementos de este módulo que otros módulos pueden usar.

```

import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ClientesRoutingModule } from './clientes-routing.module';
import { FichaClienteComponent } from './pages/ficha-cliente/ficha-cliente.component';
import { ListaClientesComponent } from './pages/lista-clientes/lista-clientes.component';
import { Router } from '@angular/router';

@NgModule({
  declarations: [
    // Componentes que contiene el Modulo (ejemplos)
    // FichaClienteComponent,
    // ListaClientesComponent
  ],
  imports: [
    CommonModule,
    // Importaciones de otros Modulos de la aplicacion

    // Si usamos componentes standalone los deberiamos incluir directamente aqui para poder exportarlos
    FichaClienteComponent,
    ListaClientesComponent
  ],
  exports: [
    FichaClienteComponent,
    ListaClientesComponent,
  ]
})
export class ClientesModule { }

```

4.2 Creación de los Modulos

- Comando: **<ng g m /path/nombreModulo>**
- Lo va a generar tomando como base a la carpeta ./app
- Modificador opcionales de utilidad

OPCION	DESCRIPCION
--routing	Generar adicionalmente un fichero de rutas propio del modulo → Gestion de lazyLoading y de rutas hijas

4.3 Comunicación entre Módulos

5 Ejercicio practico:

- Mover y/o crear nuestros componentes en una estructura de modulos.
 - Modulo para clientes
 - Crear componente page-ficha-cliente
 - Crear componente page-lista-clientes
 - Modulo para personas

6 Navegación en Angular (II). Ampliación del uso de rutas.

Ampliando el uso de rutas.

1. Paso de parámetros en las rutas
2. LazyLoading (carga de módulos cuando se necesiten)
3. Subdividir las rutas en diferentes archivos (Rutas en Modulos)

Modulo principal de rutas (app-routing.module.ts)

Definir las rutas

SECCION 3

7 Dar contenido a nuestros componentes

- Estructuración de Datos. El uso de interfaces
- Archivo .html (diseñando la interfaz)
- Archivo .ts (datos y funcionalidad)
- Definición de variables, estructuración de datos con interfaces y funciones.
- Referenciando a objetos de la clase (this)
- Directivas @for & @if

SECCION 4

Usar otros componentes y librerías de terceros

8 Apéndices.

8.4 Instalaciones recomendadas

Extensiones Visual Studio Code

Extensión	Funcionalidad
Angular Language Service	Entender sintaxis de angular
Angular Snippets	Escribir componentes y crear servicios muy rápido
Angular Schematics	Crear componentes dentro de nuestras carpetas del proyecto
Angular 2 Inline	Ver la sintaxis acorde a lo que estamos escribiendo
Auto Close Tag	Cerrar etiquetas de forma automática
Auto import	Solventa problemas con las auto-importaciones
Auto Rename Tag	Renombramiento de etiquetas (cierre)
Error Lens	Ver los errores sin necesidad de dejar el cursor encima del error
Paste JSON as Code	Interfaces acordes a objetos json
Editor Config for VSCode	Tener el editor como se recomienda por parte de angular
Better Comments	Tener los comentarios clasificados y con colores
Tailwind CSS IntelliSense	Ayuda con la gestión de estilos de Tailwind CSS
Pretty TypeScript Errors	Mejorar la visualización de errores de typeScript